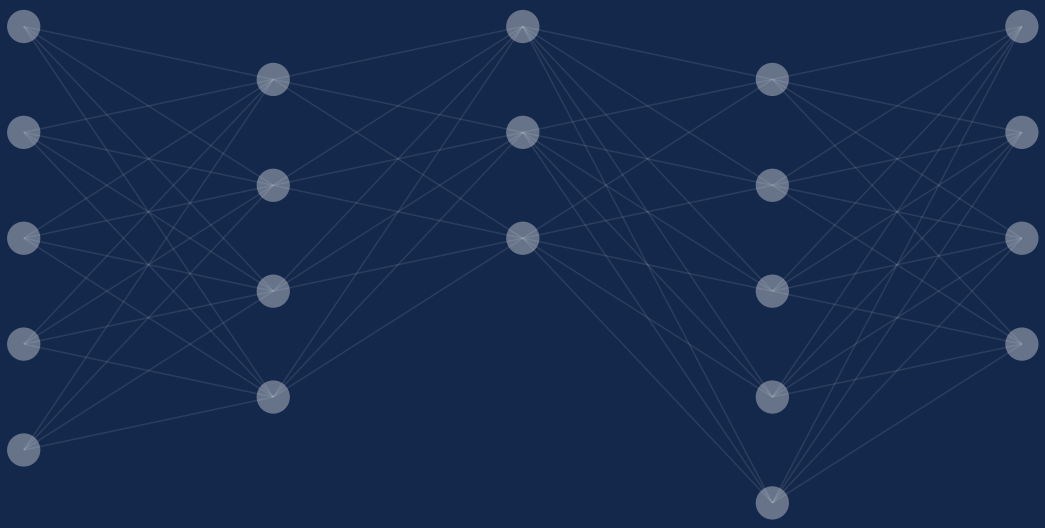


EVALUATION REPORT

AI-Based Student Performance Risk Predictor

Using an Artificial Neural Network (ANN)

TECHNOLOGY

Python · TensorFlow / Keras · Streamlit · FastAPI

DATASET

15,000 student records · 12 columns · 10 input features

TASK

3-class classification: Low · Medium · High

Contents

1 Introduction	10 Loss Function and Optimizer
2 Project Objectives	11 Model Training and Validation
3 Dataset Description	12 Model Evaluation
4 Data Preprocessing	13 Baseline Model Comparison
5 Perceptron vs Multilayer ANN	14 Model Saving and Prediction
6 ANN Architecture	15 Prediction Application and API
7 Neurons, Weights and Biases	16 Conclusion
8 Activation Functions	17 Future Improvements
9 Forward Propagation and Backpropagation	

1. Introduction

Educational institutions collect student information such as attendance, study hours, previous academic scores, assignment completion, internal assessment marks and classroom participation. Analysing these factors can help understand student academic performance and identify students who may need additional academic support.

The objective of this project is to develop an Artificial Neural Network (ANN) based prediction system that analyses academic and behavioural information and classifies students into three performance categories: **Low, Medium and High**.

The project uses Python, TensorFlow/Keras and machine learning techniques to train and evaluate the model. A Streamlit application and a FastAPI backend are also developed so that predictions can be made through a user-friendly interface.

2. Project Objectives

The main objectives are:

- To understand and analyse a structured student performance dataset.
- To perform data preprocessing and feature scaling.
- To develop a baseline machine learning model for comparison.
- To design and train a feed-forward Artificial Neural Network.
- To evaluate the ANN using classification metrics.
- To save the trained model for future predictions.
- To develop an application and API for student performance prediction.

3. Dataset Description

The project uses a structured CSV dataset containing **15,000 student records and 12 columns**. The dataset holds academic, behavioural and demographic information. The target variable is `performance_category`, which represents the student's performance class. The following ten features are used as model inputs:

Feature	Description
`age`	Age of the student
`attendance_pct`	Attendance percentage
`study_hours_per_week`	Weekly study hours
`previous_score`	Previous academic score
`assignment_completion_pct`	Assignment completion percentage
`internal_assessment_score`	Internal assessment score
`participation_score`	Classroom participation score
`sleep_hours_per_day`	Daily sleep hours
`extracurricular_hours_per_week`	Weekly extracurricular hours
`failed_subjects`	Number of failed subjects

The `student_id` column is excluded because it is only an identifier and carries no academic meaning.

Target variable: `performance_category` → the model predicts one of three classes: **Low**, **Medium** or **High**.

4. Data Preprocessing

Before training the ANN, the dataset is prepared through the following steps.

4.1 Data Inspection

The dataset is inspected to understand:

- Number of records and columns.
- Data types of the features.
- Missing values.
- Duplicate records.
- Distribution of the target classes.
- Relationships between input features.

4.2 Feature and Target Separation

The input features are separated from the target variable. The `student_id` and `performance_category` columns are left out of the input set, and the remaining ten features are used for prediction.

4.3 Label Encoding

The target variable holds text labels. Label Encoding converts these categories into numbers so that the neural network can process them.

4.4 Train-Test Splitting

The dataset is divided into **80% training data** and **20% testing data**. A random state of 42 and stratified splitting are used so that the class distribution stays the same in both sets.

4.5 Feature Scaling

StandardScaler is used to standardize the numerical input features. The standardization formula is:

$$Z = (X - mean) / standard\ deviation$$

Where:

- **X** = original value of the feature
- **mean** = average of that feature in the training data
- **standard deviation** = spread of that feature in the training data
- **Z** = standardized value

The scaler is fitted on the training data only and then applied to both training and testing data. Scaling matters because the input features have very different numerical ranges, and standardizing them helps the network train more effectively.

5. Perceptron vs Multilayer Artificial Neural Network

A perceptron is the simplest form of an artificial neural network. It generally has an input layer and a single output unit, and it is mainly used for linearly separable problems. A Multilayer ANN has an input layer, one or more hidden layers and an output layer.

Perceptron	Multilayer ANN
Basic neural network model	More advanced neural network model
Usually has no hidden layer	Contains one or more hidden layers
Learns linear decision boundaries	Can learn complex nonlinear relationships
Uses a simple learning rule	Uses backpropagation and gradient-based optimization
Limited for complex classification	Suitable for more complex classification tasks

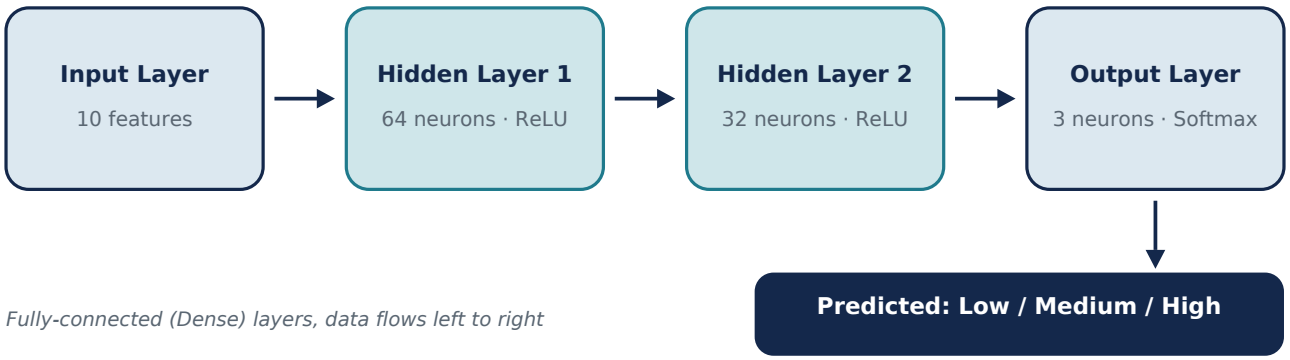
A Multilayer ANN is used in this project because student performance depends on several factors that may interact in complex ways.

6. ANN Architecture

A feed-forward Artificial Neural Network is built using TensorFlow and Keras. The architecture consists of:

1. **Input Layer** – accepts ten input features.
2. **First Hidden Layer** – 64 neurons with ReLU activation.
3. **Second Hidden Layer** – 32 neurons with ReLU activation.
4. **Output Layer** – 3 neurons with Softmax activation.

Architecture Diagram



Fully-connected (Dense) layers, data flows left to right

The architecture is designed to learn the relationship between student characteristics and their performance categories.

7. Neurons, Weights and Biases

Neurons are the basic processing units of a neural network. Each neuron takes input values, processes them and produces an output.

Weights decide how important each connection between neurons is. During training the network keeps adjusting the weights to improve its predictions.

Biases are extra learnable values that let a neuron shift its output independently of the inputs.

A neuron performs the following calculation:

$$z = (w_1 \times x_1) + (w_2 \times x_2) + \ldots + (w_n \times x_n) + b$$

Where:

- $\mathbf{x_1}, \mathbf{x_2}, \ldots, \mathbf{x_n}$ = input values
- $\mathbf{w_1}, \mathbf{w_2}, \ldots, \mathbf{w_n}$ = weights
- **b** = bias
- **z** = weighted sum

The result z is then passed through an activation function to give the neuron's final output.

8. Activation Functions

Activation functions decide the output of a neuron and bring nonlinearity into the network.

8.1 ReLU (Rectified Linear Unit)

ReLU is used in both hidden layers.

$$f(x) = max(0, x)$$

It gives 0 for negative inputs and returns the input itself for positive values. ReLU helps the network learn nonlinear relationships efficiently.

8.2 Softmax Activation

Softmax is used in the output layer because this is a three-class classification problem.

$$P(class\ i) = exp(z_i) / [\ exp(z_1) + exp(z_2) + exp(z_3)\ ]$$

Here exp(z) means e raised to the power z. It converts the three output values into class probabilities that add up to 1. The class with the highest probability becomes the final prediction.

9. Forward Propagation and Backpropagation

9.1 Forward Propagation

Forward propagation is the process in which input data moves from the input layer to the output layer. In this project:

1. The ten student features are given to the input layer.
2. The first hidden layer processes them using weights, biases and ReLU activation.
3. The second hidden layer performs further calculations.
4. The output layer applies Softmax activation.
5. The network produces probabilities for Low, Medium and High performance.

9.2 Backpropagation

Backpropagation is the learning process in which the network adjusts its weights and biases to reduce prediction errors. It involves:

1. Comparing the predicted output with the actual target.
2. Calculating the loss.
3. Computing the gradients of the loss with respect to the model parameters.
4. Passing these gradients backwards through the network.
5. Updating the weights and biases using the optimizer.

This is repeated throughout training to improve the model's performance.

10. Loss Function and Optimizer

10.1 Loss Function

The project uses **Sparse Categorical Crossentropy** as the loss function. It suits multi-class classification when the target labels are stored as integer-encoded classes, and it measures how far the predicted class probabilities are from the actual class.

10.2 Optimizer

The **Adam** optimizer updates the weights and biases during training. Adam combines the ideas of momentum and adaptive learning rates to update the parameters efficiently.

11. Model Training and Validation

The ANN is trained using the following configuration:

Parameter	Value
Framework	TensorFlow / Keras
Model Type	Feed-forward ANN
Input Features	10
Hidden Layers	2
First Hidden Layer	64 neurons
Second Hidden Layer	32 neurons
Output Neurons	3
Hidden Activation	ReLU
Output Activation	Softmax
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Epochs	Maximum 50

Parameter	Value
Batch Size	32
Validation Split	20%
Early Stopping	Patience of 7 epochs

A portion of the training data is kept aside for validation during training. Early Stopping watches the validation loss and stops training when it has not improved for seven consecutive epochs; the best weights are then restored. This saves unnecessary training and helps prevent overfitting.

12. Model Evaluation

The trained ANN is evaluated on the unseen test dataset using the metrics below.

12.1 Accuracy

Accuracy is the share of samples that are predicted correctly.

$$Accuracy = Correct\ predictions / Total\ predictions$$

12.2 Precision

Precision tells how many of the samples predicted as a particular class actually belong to that class.

$$Precision = TP / (TP + FP)$$

12.3 Recall

Recall tells how many of the actual samples of a class were correctly identified.

$$Recall = TP / (TP + FN)$$

12.4 F1-Score

The F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Here TP = true positives, FP = false positives and FN = false negatives. For this three-class problem, precision, recall and F1-score are reported using the macro average (the simple average over the three classes), along with per-class results where available.

12.5 Actual Evaluation Results

Enter the actual test-set results generated by the notebook in the table below.

Evaluation Metric	ANN Result
Accuracy	*[Insert actual value]*
Macro Precision	*[Insert actual value]*
Macro Recall	*[Insert actual value]*
Macro F1-Score	*[Insert actual value]*

12.6 Confusion Matrix

A confusion matrix compares the actual class labels with the predicted ones. It shows how many samples were classified correctly and which classes get confused with one another.

[Insert the ANN confusion matrix generated from the test dataset here]

13. Baseline Model Comparison

Logistic Regression is implemented as the baseline machine learning model. It gives a reference point for judging whether the ANN really improves on a simpler classification algorithm.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Logistic Regression	*[Insert result]*	*[Insert result]*	*[Insert result]*	*[Insert result]*
Artificial Neural Network	*[Insert result]*	*[Insert result]*	*[Insert result]*	*[Insert result]*

The comparison must use the same test dataset and the same evaluation metrics for both models.

Observation: After inserting the actual results, discuss whether the ANN performs better than Logistic Regression, whether the improvement is meaningful, and which performance categories are the hardest to classify.

14. Model Saving and Prediction

After training, the ANN is saved in Keras format as `student_performance.keras`. The preprocessing objects are saved too:

- `scaler.pkl`
- `label_encoder.pkl`

These files let the application load the trained model and make predictions without retraining each time. For a new student record, the application:

1. Receives the student's information.
2. Applies the saved StandardScaler.
3. Passes the scaled features to the trained ANN.
4. Obtains the Softmax probabilities.
5. Picks the predicted category with the highest probability.

6. Converts the encoded output back to the original class label using the saved LabelEncoder.

15. Prediction Application and API

A user-friendly prediction application is built with Streamlit. Users enter the student's age, attendance, study hours, previous score, assignment completion, internal assessment, participation, sleep hours, extracurricular hours and failed subjects. The application sends this data to a FastAPI backend through the `/predict` endpoint.

The API:

- Validates the submitted student information.
- Checks API key authentication.
- Loads and uses the saved ANN model and preprocessing objects.
- Returns the predicted performance category and class probabilities in JSON format.

The application then displays the predicted category along with the model's estimated probability for each class.

Application Screenshots

[Insert Screenshot 1: Student Information Input Form]

[Insert Screenshot 2: Prediction Result Showing Performance Category and Probabilities]

[Insert Screenshot 3: API Prediction Response, if available]

16. Conclusion

The project successfully demonstrates the workflow of building an ANN-based student performance prediction system. The dataset is analysed and preprocessed before training. A baseline Logistic Regression model and a feed-forward ANN are implemented, with the ANN using two hidden layers with ReLU activation and a Softmax output layer to classify student performance into three categories.

The model is evaluated using accuracy, precision, recall, F1-score and a confusion matrix. The trained model and preprocessing objects are saved and connected to a Streamlit application through a FastAPI backend. The project gives practical experience in data preprocessing, neural network architecture, forward propagation, backpropagation, model training, evaluation and deployment for inference.

> **Note:** The predictions are estimates based on patterns learned from the dataset. They should not be treated as definitive judgments about a student's actual academic ability or future performance.

17. Future Improvements

Possible future improvements include:

- Evaluating the model on additional real-world student datasets.
- Improving performance through hyperparameter tuning.
- Comparing additional machine learning algorithms.
- Adding visual reports and student performance trends.
- Improving model interpretability to understand the factors that influence predictions.

End of Report